



SÃO
PAULO
TECH
SCHOOL

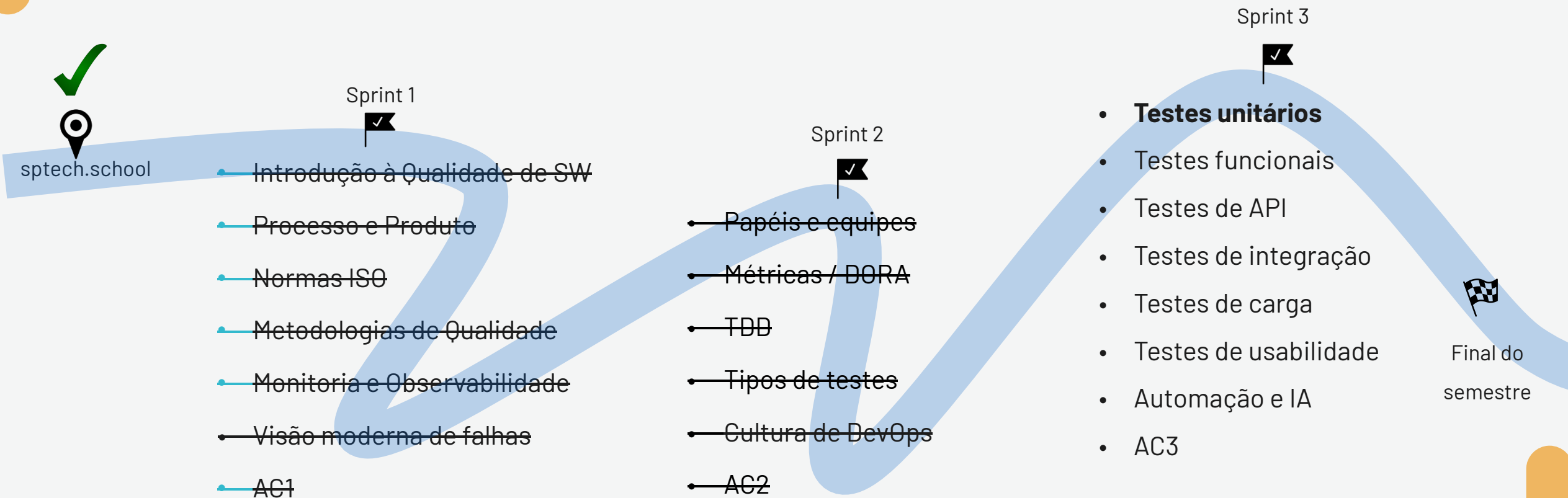
Qualidade de Software

Testes Unitários

28/abr

Prates

Nosso caminho - Ementa



Tópicos da Aula

O que são Testes Unitários?

Pirâmide de testes

Características de um bom teste unitário

Métricas e cobertura

Ferramentas e frameworks

Boas práticas

Uso de IA

Atividade

Testes Unitários

The background of the slide is a dark, textured surface. It features a series of thin, white, wavy lines that flow across the frame, creating a sense of movement and depth. These lines are interspersed with numerous small, bright white dots, which resemble stars or bokeh light effects. The overall aesthetic is modern and tech-oriented.



O que são Testes Unitários?

- Verificação automática de unidades isoladas de código (funções, métodos)
- Detectam defeitos cedo → menor custo de correção
- Base do desenvolvimento orientado a testes (TDD)



Importância dos testes unitários

- Previnem erros em regressões
- Garantem comportamento esperado
- Melhoram design e modularidade do código
- Aumentam confiança no refactoring
- Reduzem esforço de manutenção

Pirâmide de testes





Características de um bom teste unitário

- Rápido
- Pequeno (responsabilidade clara e definida)
- Determinístico (sempre mesmo resultado)
- Independente
- Automático
- Legível
- Modularizar testes (separado do código)
- Reuso de código com base no tipo de teste (por exemplo, formulários, CRUD, etc)



Métricas e cobertura

- Cobertura de linhas
- Cobertura de branches
- Cobertura de funções/métodos
- Ferramentas reportam porcentagem de código testado.
- 🙌 Importante: Alta cobertura não garante ausência de bugs, mas ausência de cobertura é um alerta claro.



Ferramentas e Frameworks

- Python:
 - unittest
 - pytest
 - nose2
- Java: JUnit
- JavaScript: Jest, Mocha
- C#: xUnit, NUnit



Ferramentas (cobertura)

- Python:
 - coverage.py (mais usado)
 - pytest-cov



Boas práticas no uso de testes unitários

- Escrever testes pequenos e específicos
- Nomear bem os testes → documentação viva
- Mockar dependências externas
- Rodar os testes em CI/CD
- Refatorar testes também!



Uso de IA na geração de testes

- Modelos de IA conseguem:
 - Analisar código existente
 - Gerar templates de testes unitários
 - Sugerir casos de borda
- Geração automatizada reduz tempo, mas exige validação humana!



Exemplo

- Exemplo de prompt para IA:
 - "Gere testes unitários usando pytest para a seguinte função Python: def is_even(n): return n % 2 == 0"
- Exemplo gerado:

python

```
def test_is_even():  
    assert is_even(2) == True  
    assert is_even(3) == False
```




Limitações do uso de IA

- Testes simples → IA é ótima
- Testes complexos, mocks, dependências externas → IA ainda precisa de apoio humano
- Não substitui raciocínio crítico do engenheiro de testes

The background is a dark, almost black, space filled with intricate, glowing patterns. Fine, white, wavy lines flow across the frame, creating a sense of movement and depth. Interspersed among these lines are numerous small, bright white dots and larger, soft, out-of-focus circles (bokeh), which add to the ethereal and dynamic feel of the image.

Atividade

Exemplo - sistema de caixa eletrônico



Exemplo - código de validação de credenciais

```
# arquivo: autenticacao.py

def validar_credenciais(usuario, senha, base_dados):
    """
    Valida se o usuário e senha existem na base de dados simulada.

    base_dados: dict com usuários e senhas (ex.: {'user1': 'senha123'})
    """
    if usuario in base_dados and base_dados[usuario] == senha:
        return True
    return False

# Exemplo de uso (simulado)
if __name__ == "__main__":
    usuarios = {'joao': 'abc123', 'maria': 'senhaSegura'}
    print(validar_credenciais('joao', 'abc123', usuarios)) # True
    print(validar_credenciais('joao', 'errada', usuarios)) # False
```

Exemplo - código de operação de saque

```
# arquivo: operacoes.py

def realizar_saque(saldo_atual, valor_saque):
    """
    Realiza um saque se houver saldo suficiente.
    """
    if valor_saque <= 0:
        raise ValueError("Valor de saque inválido")
    if valor_saque > saldo_atual:
        raise ValueError("Saldo insuficiente")
    return saldo_atual - valor_saque

# Exemplo de uso (simulado)
if __name__ == "__main__":
    print(realizar_saque(500, 100)) # 400
    # print(realizar_saque(500, 600)) # Levanta exceção
```




Exercício

- Construir testes unitários para os códigos acima, testando as seguintes condições:
- Credenciais: sucesso e falha na autenticação, usuário inexistente, tentativas de autenticar excedidas
- Saque: saque normal, saldo insuficiente, saque zero, limite diário excedido



Próxima aula

Testes funcionais



Agradeço a sua atenção!

Qualidade de Software

Professor Prates

Aula 9

SÃO
PAULO
TECH
SCHOOL